UCAM

Tema 2

Introducción a kotlin

Aplicaciones para Dispositivos Móviles

Daniel Valero Carreras

Grado en Ingeniería Informática



ÍNDICE

CONTENIDOS

1. ¿Qué es kotlin?
2. Componentes kotlin

Imagen 1

1. ¿Qué es Kotlin?



Esta foto de Autor desconocido está bajo licencia CC BY-SA-NC

1. ¿Qué es kotlin?
Características

- Lenguaje moderno de programación creado por JetBrains (2011).
- Totalmente interoperable con Java (puede usar librerías y código Java).
- Oficial para desarrollo Android (desde 2017, según Google).
- Compila a JVM, pero también a JavaScript y nativo (Kotlin/Native).
- Sintaxis simple, segura y expresiva → menos código, menos errores.
- Seguridad frente a nulls (null safety) para evitar errores comunes.
- Soporta programación funcional y orientada a objetos.
- Compatible con frameworks modernos (Spring, Compose, etc.).
- Usado en Android, backend, web y multiplataforma (Kotlin Multiplatform)

2. Componentes de Kotlin

Imagen 7
Pantalla con herramientas

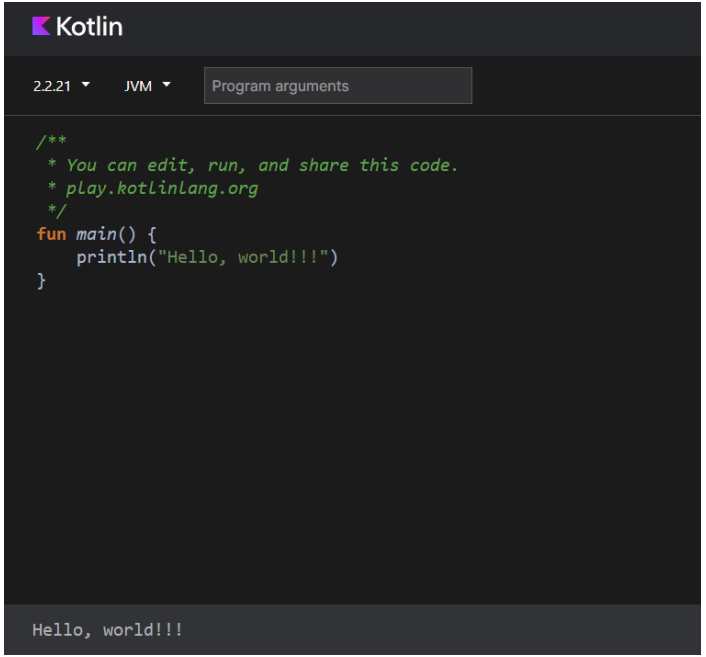


Nota. Pantalla de la que salen iconos de herramientas

2. Componentes de Kotlin
Hello world

- ✓ Editor de código kotlin online:
  - ✓ https://play.kotlinlang.org/

```
fun main() {
    println("Hola mundo")
}
```



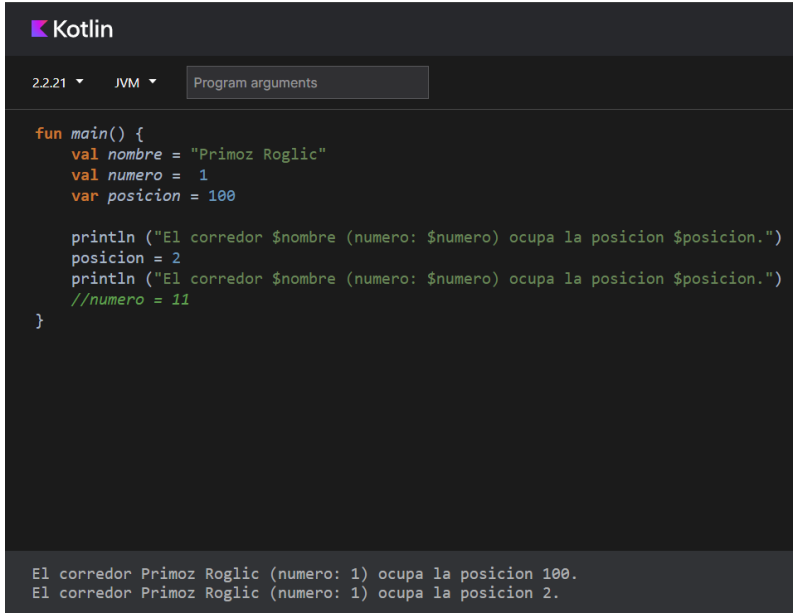
2. Componentes de Kotlin

Var y val

- ✓ Var: es una variable que puede cambiar de valor
- ✓ Val: es una variable de solo lectura
- ✓ Para hacer referencia a ellas usamos $

```
fun main() {
    val nombre = "Primoz Roglic"
    val numero =  1
    var posicion = 100

    println ("El corredor $nombre (numero: $numero) ocupa la posicion $posicion.")
    posicion = 2
    println ("El corredor $nombre (numero: $numero) ocupa la posicion $posicion.")
    //numero = 11
}
```



2. Componentes de Kotlin

Ejercicio

- ✓ Ejercicio: Crea un programa con diferentes variables como: nombre, equipo, número y posición, que muestre por pantalla el siguiente mensaje:

  "El corredor Carlos Rodriguez (numero:11) del equipo Ineos ocupa la posición 5."

2. Componentes de Kotlin

Tipos de variables

- ✓ Integers: Byte, Short, Int, Long
- ✓ Unsigned integers: UByte, UShort, UInt, Ulong
- ✓ Floating-point numbers: Float, Double
- ✓ Booleans: Boolean
- ✓ Characters: Char
- ✓ Strings: String

```
fun main() {
    val nombre: String = "Primoz Roglic"
    val numero: Int =  1
    var posicion: Int = 100
    val v_sprint: Float = 69.5f

    println ("El corredor $nombre (numero: $numero) ocupa la posicion $posicion." +
            "Tiene un valor de sprint de $v_sprint")
    posicion = 2

}
```

2. Componentes de Kotlin

Tipos de variables

Kotlin

2.2.21 JVM Program arguments

```
fun main() {
    val nombre: String = "Primoz Roglic"
    val numero: Int =  1
    var posicion: Int = 100
    val v_sprint: Float = 69.5f

    println ("El corredor $nombre (numero: $numero) ocupa la posicion $posicion." +
            "Tiene un valor de sprint de $v_sprint")
    posicion = 2

}
```

```
El corredor Primoz Roglic (numero: 1) ocupa la posicion 100.Tiene un valor de sprint de 69.5
```

2. Componentes de Kotlin

Colecciones

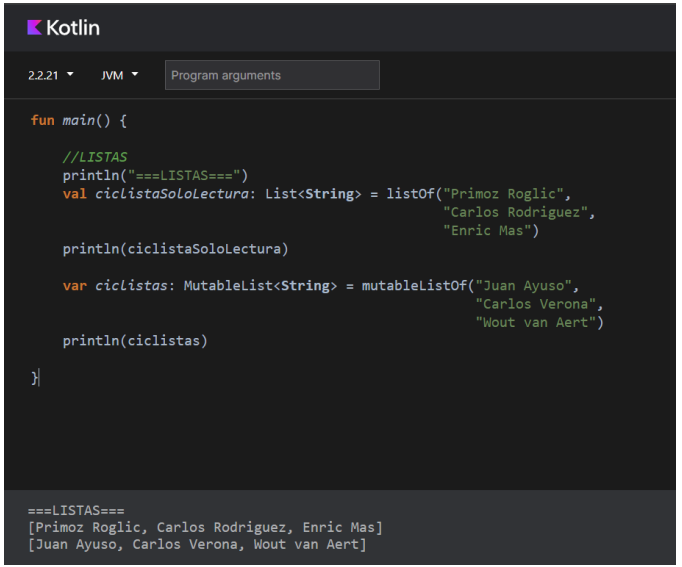
- ✓ Lists: Colección ordenada de elementos
  - ✓ De lectura: no se pueden modificar los elementos-> List<> -> listOf()
  - ✓ Variables: se pueden modificar los elementos -> MutableList<> -> mutableListOf()
- ✓ Sets: Colección desordenada de elementos. Solo almacenan elementos ÚNICOS
  - ✓ De lectura: no se pueden modificar los elementos-> Set<> -> setOf()
  - ✓ Variables: se pueden modificar los elementos -> MutableSet<> -> mutableSetOf()
- ✓ Maps: conjuntos de clave-valor. Las claves son únicas y tienen un valor
  - ✓ De lectura: no se pueden modificar los elementos-> Map<> -> mapOf()
  - ✓ Variables: se pueden modificar los elementos -> MutableMap<> -> mutableMapOf()

2. Componentes de Kotlin

Colecciones: listas

```
fun main() {
    println("===LISTAS===")
    val ciclistaSoloLectura: List<String> = listOf("Primoz Roglic", "Carlos Rodriguez", "Enric Mas")
    println(ciclistaSoloLectura)

    var ciclistas: MutableList<String> = mutableListOf("Juan Ayuso", "Carlos Verona", "Wout van Aert")
    println(ciclistas)
}
```



2. Componentes de Kotlin

Colecciones: conjuntos

```
fun main() {
println("===SETS===")
    val ciclistaSoloLecturaSet: Set<String> = setOf( "Carlos Rodriguez", "Primoz Roglic", "Carlos Rodriguez", "Enric Mas")
    println(ciclistaSoloLecturaSet)

    var ciclistasSet: MutableSet<String> = mutableSetOf("Juan Ayuso", "Carlos Verona", "Wout van Aert")
    println(ciclistasSet)
}
```

Kotlin

2.2.21 JVM Program arguments

```
fun main() {
    println("===SETS===")
    val ciclistaSoloLecturaSet: Set<String> = setOf( "Carlos Rodriguez",
                                                     "Primoz Roglic",
                                                     "Carlos Rodriguez",
                                                     "Enric Mas")
    println(ciclistaSoloLecturaSet)

    var ciclistasSet: MutableSet<String> = mutableSetOf("Juan Ayuso",
                                                        "Carlos Verona",
                                                        "Wout van Aert")
    println(ciclistasSet)
}
```

```
===SETS===
[Carlos Rodriguez, Primoz Roglic, Enric Mas]
[Juan Ayuso, Carlos Verona, Wout van Aert]
```

2. Componentes de Kotlin

Colecciones: maps

```
fun main() {

println("===MAPS===")

    val ciclistasEquiposMap: Map<String, String> =
mapOf("Carlos Rodriguez" to "Ineos", "Primoz Roglic" to
"Jumbo-Visma", "Enric Mas" to "Movistar")

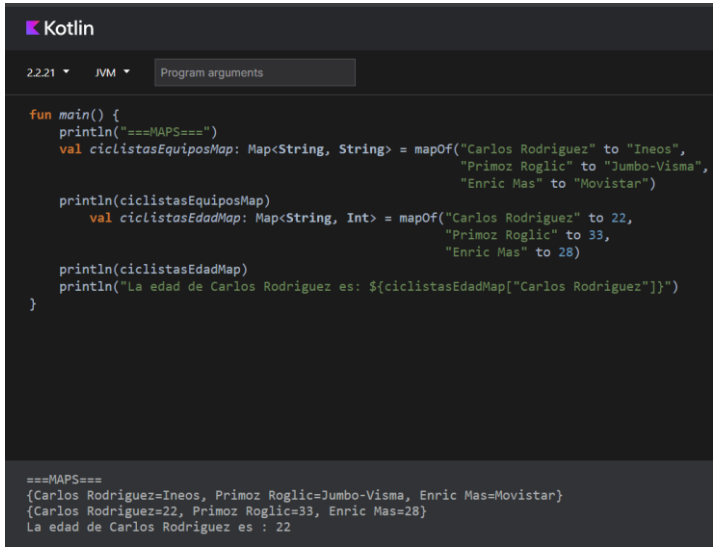
    println(ciclistasEquiposMap)

        val ciclistasEdadMap: Map<String, Int> =
mapOf("Carlos Rodriguez" to 22, "Primoz Roglic" to 33,
"Enric Mas" to 28)

    println(ciclistasEdadMap)

    println("La edad de Carlos Rodriguez es :
${ciclistasEdadMap[1]}")}

}
```



2. Componentes de Kotlin

Flujos de control: if/else

```
val edad = 18
if (edad >= 18) {
    println("Eres mayor de edad")
} else {
    println("Eres menor de edad")
}

val mensaje = if (edad >= 18) "Adulto" else "Menor"
```

2. Componentes de Kotlin

Flujos de control: switch

```
val dia = 3


when (dia) {
    1 -> println("Lunes")
    2 -> println("Martes")
    3 -> println("Miércoles")
    in 4..5 -> println("Jueves o Viernes")
    else -> println("Fin de semana")
}
```

2. Componentes de Kotlin

Flujos de control: for

```
for (i in 1..5) {
    println(i)
}
for (i in 10 downTo 1) println(i)      // Cuenta regresiva
for (i in 0 until 10 step 2) println(i) // Salta de 2 en 2
```

```
val clasificacion: Map<String, Int> = mapOf("Carlos Rodriguez" to 1, "Primoz Roglic" to 3, "Enric Mas" to 6)
 for (ciclista in clasificacion){
        if (ciclista.value == 1){
            println("El ciclista ${ciclista.key} es el lider")
        }else{
            println("El ciclista ${ciclista.key} ocupa la posición ${ciclista.value}")
        }
}
```

2. Componentes de Kotlin

Flujos de control: foreach

```
val frutas = listOf("Manzana", "Banana", "Pera")


frutas.forEach { fruta ->
    println(fruta)
}
val numeros = listOf(10, 20, 30)
numeros.forEachIndexed { index, valor ->
    println("Índice $index tiene valor $valor")
}
```

2. Componentes de Kotlin

Funciones en kotlin

```
fun nombreDeFuncion(parametros): TipoDeRetorno {
    // cuerpo de la función
    return valor
}
//Ejemplo
import kotlin.random.Random
fun correrCarrera(): Int {
    var posicion = Random.nextInt(0, 190)
    return posicion
}
```

2. Componentes de Kotlin
Funciones

- ✓ Funciones in-line: Si la función tiene **una sola instrucción**, se puede escribir más corta:

```
fun asignarNumero(): Int = Random.nextInt(0, 190)}
```

- ✓ Funciones sin valor de retorno (Unit): como el void en C

```
fun saludar(nombre: String): Unit {
    println("Hola, $nombre")
}
```

2. Componentes de Kotlin

Funciones

- ✓ Funciones lambda: Una **lambda** (también llamada **función anónima**) es **una función sin nombre**, que puedes **definir y usar como valor**.
- ✓ { parametros -> cuerpo }

```
val cuadrado = { x: Int -> x * x }
println(cuadrado(4))  // Salida: 16
```

- ✓ Se pueden utilizar como argumento de funciones:

```
fun operar(a: Int, b: Int, operacion: (Int, Int) -> Int): Int {
    return operacion(a, b)
}
val suma = operar(5, 3,{ x, y -> x + y })
```

2. Componentes de Kotlin

Clases

- ✓ Parecidas a Java

```
class Persona(val nombre: String, var edad: Int) {
    fun saludar() {
        println("Hola, soy $nombre y tengo $edad años.")
    }
}
val persona = Persona(“Paquito", 75)
persona.saludar()
```

2. Componentes de Kotlin

Clases: herencia

- ✓ Por defecto, las clases son **final** (no se pueden heredar). Para ello es necesario open
- ✓ No se puede múltiple pero sí de varios interfaces

```
open class Persona(val nombre: String, var edad: Int) {
    open fun saludar() {
        println("Hola, soy $nombre y tengo $edad años.")
    }
}
class Ciclista(nombre: String, edad: Int): Persona(nombre, edad){
   override fun saludar() = println("Soy el ciclista $nombre")
}
```

2. Componentes de Kotlin

Clases: herencia

```
fun main(){
   val ciclista: Ciclista = Ciclista("Dani", 45)
   val persona: Persona = Persona("Antonio Jesus", 65)
   ciclista.saludar()
   persona.saludar()
}
```

Kotlin

2.2.21 JVM Program arguments

```
open class Persona(val nombre: String, var edad: Int) {
    open fun saludar() {
        println("Hola, soy $nombre y tengo $edad años.")
    }
}

class Ciclista(nombre: String, edad: Int): Persona(nombre, edad){
   override fun saludar() = println("Soy el ciclista $nombre")
}

fun main(){
   val ciclista: Ciclista = Ciclista("Dani", 45)
   val persona: Persona = Persona("Antonio Jesus", 65)

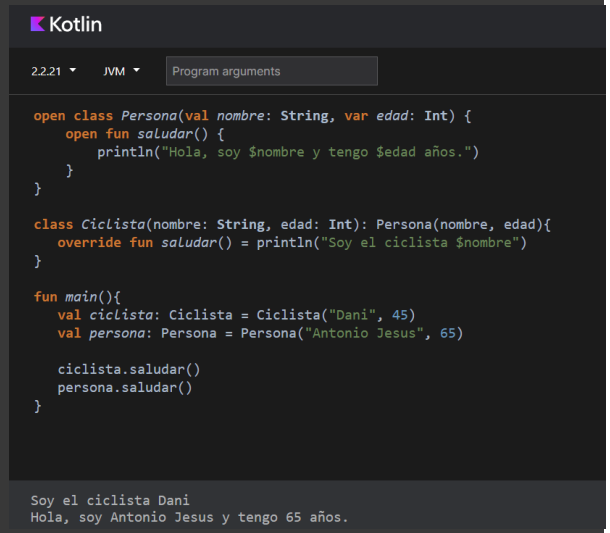
   ciclista.saludar()
   persona.saludar()
}
```

```
Soy el ciclista Dani
Hola, soy Antonio Jesus y tengo 65 años.
```

2. Componentes de Kotlin

Clases: herencia

```
fun main(){
   val ciclista: Ciclista = Ciclista("Dani", 45)
   val persona: Persona = Persona("Antonio Jesus", 65)
   ciclista.saludar()
   persona.saludar()
}
```



2. Componentes de Kotlin
data classes

- ✓ Son un tipo especial de clase pensada para **almacenar información** de manera simple, sin tener que escribir mucho código repetitivo.
- ✓ **Genera** métodos útiles como:
  - ✓ toString() → para imprimir el contenido legiblemente
  - ✓ equals() y hashCode() → para comparar objetos por **valor**, no por referencia
  - ✓ copy() → para clonar objetos cambiando algunas propiedades
  - ✓ componentN() → para hacer **desestructuración**

2. Componentes de Kotlin

data classes

```
data class Ciclista(val nombre: String, val age: Int, val sprint: Int, val equipo: String){}
fun main() {
    val ciclista = Ciclista("Dani",33, 66, "UCAM")
    println(ciclista.toString())
    val ciclista2 = Ciclista("Claudio Chiapucci", 56, 66, "Carrera")
    println(ciclista.equals(ciclista2))
    val ciclista3 = ciclista.copy()
    println(ciclista.equals(ciclista3))
    val (nombre,_,sprint) = ciclista
    println(nombre)
    println(sprint)
}
```

2. Componentes de Kotlin
Clases abstractas

- ✓ No se pueden instanciar
- ✓ Se utilizan para que todos los elementos que hereden de esa clase tengan los mismos métodos

```
abstract class Animal {
    abstract fun sonido()
}
class Gato (nombre: String, edad: Int) : Animal() {
   override fun sonido(){
      println("El gato hace miau.")
   }
}
```

```
class Perro (nombre: String, edad: Int) : Animal (){
override fun sonido(){
      println("El perro hace guau.")
   }
}}
fun main() {
   val gato = Gato("Garfield",10)
   gato.sonido()
   val perro = Perro("Scooby",7)
   perro.sonido()}
```

2. Componentes de Kotlin
Interface

- ✓ Un **interfaz** es un **contrato** que especifica *qué* debe hacer una clase, sin imponer *cómo* debe hacerlo.
- ✓ Define un conjunto de métodos y propiedades que una clase debe implementar

```
interface ParticiparCarrera {
    val nombreCarrera: String
    fun atacar(){
        println("El ciclista está atacando.")
    }
    fun sprintar()
}
class Ciclista (nombre: String, apellido: String, edad: Int, equipo: String, val sprint: Int): Persona(nombre,apellido,edad,equipo), ParticiparCarrera {
    override val nombreCarrera: String = "Tour de Francia"
    override fun sprintar(){
        println("El ciclista $nombre $apellido está sprintando ($sprint) en el $nombreCarrera")
    }
}
```

2. Componentes de Kotlin
Coroutines

- ✓ Las **coroutines** (corrutinas) son una forma **simple y eficiente de manejar tareas asíncronas o concurrentes** en Kotlin.
- ✓ En lugar de usar **threads** complicados, las coroutines permiten escribir **código asíncrono como si fuera secuencial**.

2. Componentes de Kotlin

Ejercicio

- ✓ Crea una clase que se llame estudiante y tenga como atributos: nombre y nota
- ✓ Crea una función que se llame mostrarEstudiantes y se le pase por parámetro una lista de estudiantes
  - ✓ Dentro de la función, se comprueba la nota del alumno y si es inferior a 5 se muestra el mensaje “suspenso”. Al revés “aprobado”
- ✓ Crea una lista de alumnos y prueba las funciones anteriores

REFERENCIAS

BIBLIOGRÁFICAS

Kotling: https://kotlinlang.org/

Play kotling lang: *https://play.kotlinlang.org/*

Daniel Valero Carreras

dvalero@ucam.edu

UCAM Universidad Católica de Murcia



© UCAM